

AWS VPC deployment plan — SugamFlow

Document version: 1.1 — SugamFlow (sugamflow.com, Angular apps, Spring Cloud microservices).

This plan maps the repository stack (config-server, Eureka discovery, API gateway, domain services) to AWS networking and compute patterns suitable for production.

1. High-level workload placement

Marketing site (sugamflow.com / sugamflow-landing): S3 static hosting + CloudFront. Optional: keep existing Vercel/Firebase; point DNS accordingly.

Angular shop app (shop-management-ui): S3 + CloudFront or Amplify Hosting. Configure production environment so API base URL points to the public API host (ALB), not individual microservice ports.

Spring Cloud Gateway: the only user-facing HTTP API entry from the internet — Application Load Balancer (ALB) in public subnets !' gateway-service (container port 9090) in private subnets.

Config server, Eureka (discovery), and all backend microservices: run only in private subnets with no direct public IPs. Internal service-to-service traffic stays inside the VPC.

SQL Server: RDS for SQL Server (or self-managed SQL on EC2 only if required) in private database subnets. Replace development host.docker.internal with the RDS endpoint and store credentials in AWS Secrets Manager.

2. VPC topology (single region, production-oriented)

Use one VPC, for example 10.0.0.0/16, across three Availability Zones for high availability.

Public subnets (e.g. /24 per AZ): Internet Gateway; NAT Gateway per AZ (recommended for production resilience); Application Load Balancer for HTTPS to the API gateway.

Private application subnets: ECS Fargate tasks or EKS worker nodes hosting — config-service (8888), discovery-service (8761), gateway-service (9090), shop, product, stock, order, user, auth, payment, notification, reporting, account services (ports 8080–8089 per compose).

Private database subnets: RDS for SQL Server — no public accessibility.

Optional: VPC interface endpoints (ECR, Secrets Manager, CloudWatch Logs, SSM) to reduce NAT dependency and tighten egress controls.

3. Traffic flow (summary)

End users !' CloudFront !' S3: static assets for the marketing site and the Angular application.

End users !' api.sugamflow.com !' ALB (HTTPS) !' gateway-service !' downstream microservices via load-balanced internal names (Eureka / ECS Service Discovery / Cloud Map).

Microservices !' config-service and discovery-service: VPC-internal only.

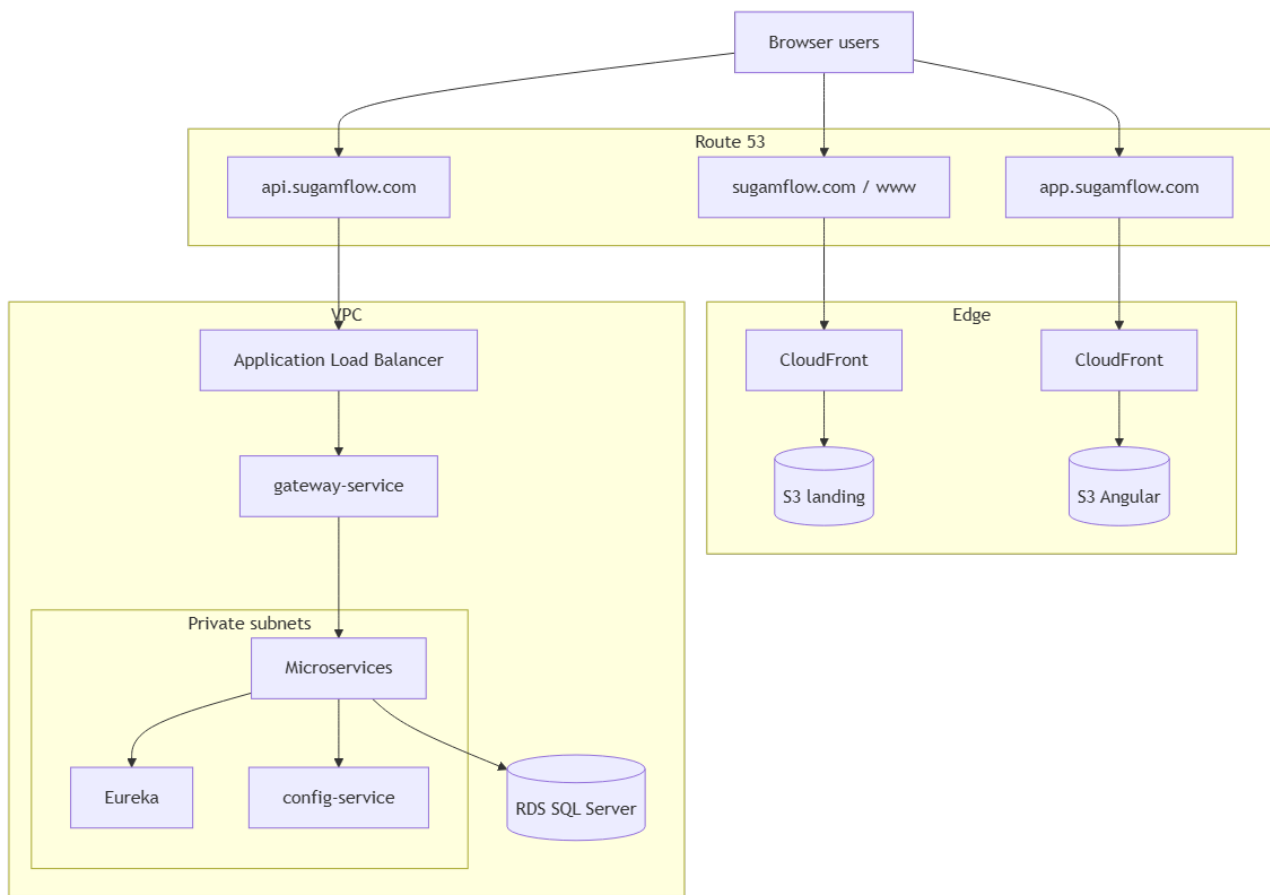
Microservices !' RDS: SQL Server port 1433 from application security groups only.

Do not expose Eureka (8761), config (8888), or individual REST ports on a public load balancer; only the gateway should be internet-facing for API traffic.

Architecture flow diagrams (sugamflow.com)

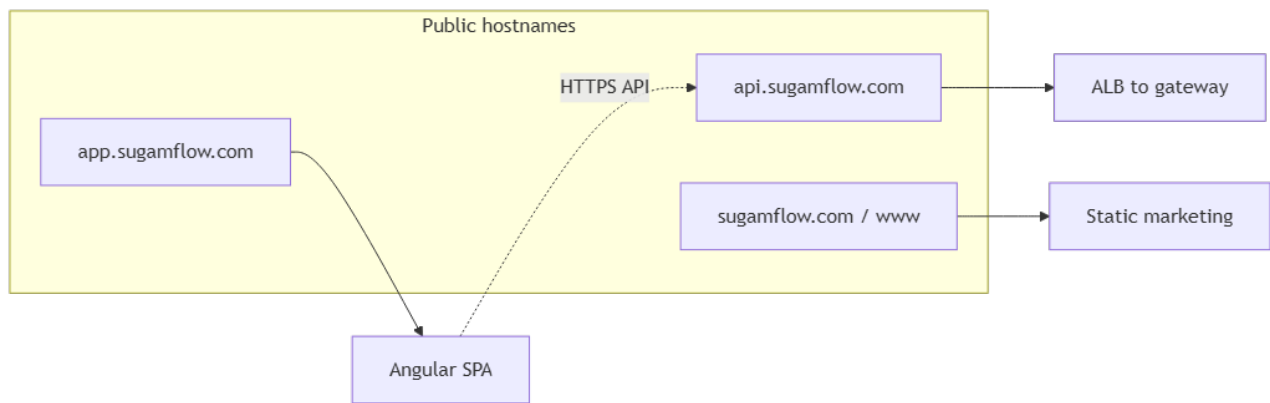
The following figures summarize DNS, edge, and VPC paths. Source: docs/diagrams/*.mmd — regenerate PNG with `npm run diagrams`.

Flow diagram A — End-to-end (domain: sugamflow.com)



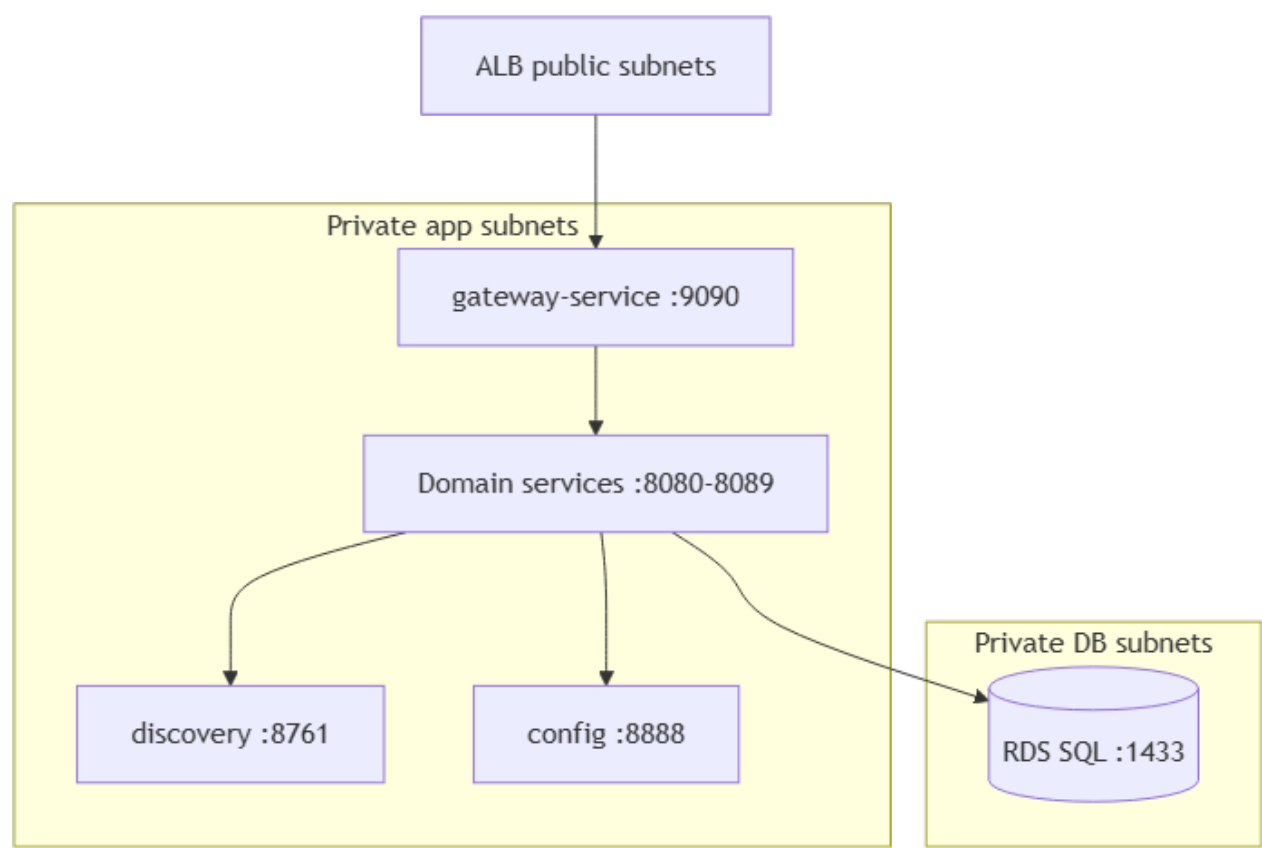
Users, Route 53 records, CloudFront + S3 for marketing and app static files, and `api.sugamflow.com` !' ALB !' gateway !' internal services and RDS.

Flow diagram B — Public hostname roles



sugamflow.com / www serves marketing; app.sugamflow.com serves the Angular SPA; api.sugamflow.com terminates HTTPS at the ALB and reaches only the gateway.

Flow diagram C — VPC internal (east-west)



ALB to gateway in private subnets; microservices use Eureka, config-server, and RDS without public exposure.

4. Security groups (minimal model)

alb-sg: inbound TCP 443 from the internet (or restrict to CloudFront/origin patterns if applicable).

gateway-sg: inbound from alb-sg only, on the container port for gateway-service.

internal-ms-sg: inbound from gateway-sg and peer microservices as required — restrict ports (8080–8089, 8761, 8888) to least privilege.

rds-sg: inbound TCP 1433 only from internal application security groups.

5. Compute platform

Recommended default: Amazon ECS on AWS Fargate + ALB — aligns with your Docker images and reduces operational overhead for a Spring Cloud set of this size.

Alternative: Amazon EKS if you standardize on Kubernetes or need a service mesh (e.g. Istio) later.

Service discovery: you may keep Spring Cloud Eureka initially to minimize code changes; consider AWS Cloud Map / ECS Service Connect over time for simpler operations.

6. Configuration, secrets, and bootstrap order

Spring Cloud Config: back the repo with CodeCommit, S3, or private GitHub; restrict access with IAM; run config-service reachable only inside the VPC.

Secrets: AWS Secrets Manager or SSM Parameter Store for `SPRING_DATASOURCE_*`, `SECURITY_JWT_SECRET`, mail credentials, and integration keys — inject into task definitions.

Startup dependency order mirrors local Docker Compose: config !' discovery !' gateway and domain services; use ECS health checks and `depends_on` semantics.

7. DNS and TLS (sugamflow.com)

Amazon Route 53: sugamflow.com and www !' CloudFront for static sites; app.sugamflow.com !' CloudFront for the Angular app; api.sugamflow.com !' ALB.

AWS Certificate Manager: certificates in us-east-1 for CloudFront; in the application region for the ALB.

8. Observability

Amazon CloudWatch Logs per ECS service; optional AWS X-Ray from gateway through downstream calls.

Auto scaling on gateway and stateless services; RDS Multi-AZ when budget allows.

9. Service inventory (from docker-compose)

Infrastructure: config-service, discovery-service, gateway-service.

Domain: shop-service, product-service, stock-service, order-service, user-service, auth-service, payment-service, notification-service, reporting-service, account-service.

Frontends: static builds for sugamflow-landing and shop-management-ui served via S3/CloudFront; CORS and environment URLs must target the public API host through the gateway.

10. Suggested implementation sequence

- 1) Finalize CIDR and subnet layout; choose ECS Fargate vs EKS.
- 2) Provision RDS for SQL Server and migrate connection configuration.
- 3) Push container images to Amazon ECR; define task definitions per service.
- 4) Create ALB target group pointing only to gateway-service; lock security groups.
- 5) Set Angular production environment to <https://api.sugamflow.com> (through gateway).